Implement a simple Tic-Tac-Toe Game using Reinforcement Learning

Aim

To write a program to implement a simple Tic-Tac-Toe game using Reinforcement Learning.

Algorithm:

1. Initialize a 3×3 Tic-Tac-Toe board with empty cells (0).
2. Initialize two Q-learning agents (X and O) with Q-tables and learning parameters.
3. Training Phase:
    - Identify available moves.
    - Select a move using the epsilon-greedy strategy.
    - Update the board and switch players.
    - Assign rewards: +1 for a win, 0.5 for a draw, -1 for a loss.
    - Update the Q-table using the Q-learning formula.
4. Testing Phase:
    - Reset the board and load trained Q-tables.
    - Players take turns selecting moves using trained Q-values.
    - The game continues until a winner is found or the board is full.
    - Print the final board and declare the result.

Program

```
import numpy as np import

random class TicTacToe:

    def __init__(self):

        self.board = np.zeros((3, 3))                    # 3x3 board initialized to 0 self.done =

        False

        self.winner = None
```

```
def reset(self):
        self.board = np.zeros((3, 3)) self.done = False
        self.winner = None return
        self.get_state()
def get_state(self):
        return tuple(self.board.flatten()) def
is_valid_move(self, row, col):
        return self.board[row, col] == 0 def
make_move(self, row, col, player):
        if self.is_valid_move(row, col): self.board[row,
                col] = player if self.check_winner(player):
                        self.done = True self.winner = player
                elif np.all(self.board != 0): self.done = True    # Game is
                        a draw
                return True return False
def check_winner(self, player): for i in range(3):
                if np.all(self.board[i, :] == player) or np.all(self.board[:, i] == player):
                    return True
        if np.all(np.diag(self.board) == player) or np.all(np.diag(np.fliplr(self.board)) == player):
                return True return False
```

```
def print_board(self):
        symbol_map = {1: 'X', -1: 'O', 0: ' '} print("\nBoard:")
        for row in self.board:
                print('|'.join([symbol_map[int(cell)] for cell in row])) print("-" * 5)
class QLearningAgent:
def __init__(self, player, alpha=0.1, gamma=0.9, epsilon=0.1): self.q_table = {}        #
        Initialize Q-table as a dictionary self.alpha = alpha                # Learning rate
        self.gamma = gamma             # Discount factor self.epsilon =
        epsilon                        # Exploration rate
        self.player = player                # Agent identifier (1 for X, -1 for O) def
choose_action(self, state, available_actions):
        if random.uniform(0, 1) < self.epsilon: return
                random.choice(available_actions)
          else:
                q_values = [self.q_table.get((state, action), 0) for action in available_actions]
                max_q_value = max(q_values)
                max_q_actions = [action for action, q_value in zip(available_actions, q_values) if q_value
== max_q_value]
                return random.choice(max_q_actions)
    def update_q_table(self, state, action, reward, next_state, available_actions):
        current_q = self.q_table.get((state, action), 0) if available_actions:
                next_q_values = [self.q_table.get((next_state, a), 0) for a in available_actions]
```

```
                max_next_q = max(next_q_values) else:
                max_next_q = 0
            self.q_table[(state, action)] = current_q + self.alpha * (reward + self.gamma * max_next_q -
current_q)
def train_tic_tac_toe(episodes=10000): game =
    TicTacToe()
    agent_X = QLearningAgent(player=1) agent_O =
    QLearningAgent(player=-1) agents = {1: agent_X, -
    1: agent_O} for _ in range(episodes):
        state = game.reset()
        player = 1          # Start with player X while not
        game.done:
            available_actions = [(r, c) for r in range(3) for c in range(3) if game.is_valid_move(r, c)]
            agent = agents[player]
            action = agent.choose_action(state, available_actions)
            game.make_move(action[0], action[1], player) next_state = game.get_state()
            available_next_actions = [(r, c) for r in range(3) for c in range(3) if game.is_valid_move(r,
c)]
            if game.done:
                if game.winner == player: reward = 1
                elif game.winner is None:                #Draw
                    reward = 0.5
                else:
                    reward = -1
```

else:

reward = 0

agent.update_q_table(state, action, reward, next_state, available_next_actions)

state = next_state

player = -player # Switch players return

agent_X.q_table, agent_O.q_table

q_table_X, q_table_O = train_tic_tac_toe(episodes=10000) def test_tic_tac_toe():

game = TicTacToe()

agent_X = QLearningAgent(player=1) agent_O =

QLearningAgent(player=-1) agent_X.q_table =

q_table_X agent_O.q_table = q_table_O

agents = {1: agent_X, -1: agent_O} state = game.reset()

player = 1 # Start with player X while not

game.done:

game.print_board() # Print the board after each move

available_actions = [(r, c) for r in range(3) for c in range(3) if game.is_valid_move(r, c)]

agent = agents[player]

action = agent.choose_action(state, available_actions) game.make_move(action[0],

action[1], player)

state = game.get_state()

player = -player # Switch players

print("Final Board:") game.print_board()

```
    if game.winner:
        print(f"Player {game.winner} ({'X' if game.winner == 1 else 'O'})
wins!")
    else:
        print("It's a draw!") test_tic_tac_toe()
```

OUTPUT

```
Board:
  | |
--------
  | |
--------
  | |
Board:
X| |
--------
  | |
--------
  | |
Board:
X| |
--------
  |O|
--------
  | |
Board:
X| |
--------
```

|O|

X| |

Board:

X| |

|O|

X|O|

Board:

X| |

X|O|

X|O|

Final Board:

X| |

X|O|

X|O|

Player 1 (X) wins!

Implement a 10-armed Gradient Bandit Algorithm

Aim

To implement a 10-armed gradient bandit algorithm.

Algorithm

1. Set the number of arms (*k*), learning rate (*alpha*), action preferences, and uniform probabilities.
2. Compute action probabilities using the softmax function.
3. Select an action based on these probabilities.
4. Adjust action preferences using the reward baseline and softmax probabilities.
5. Run multiple trials, selecting actions and updating preferences at each step.
6. Compute and plot the average reward over time.

Program

import numpy as np

import matplotlib.pyplot as plt class

GradientBandit:

def __init__(self, k=10, alpha=0.1): self.k = k #

Number of arms

self.alpha = alpha # Step size (learning rate) self.preferences = np.zeros(k) #

Preferences for each action

self.probabilities = np.ones(k) / k # Initial uniform probabilities

self.average_reward = 0 # Baseline for the reward def

softmax(self):

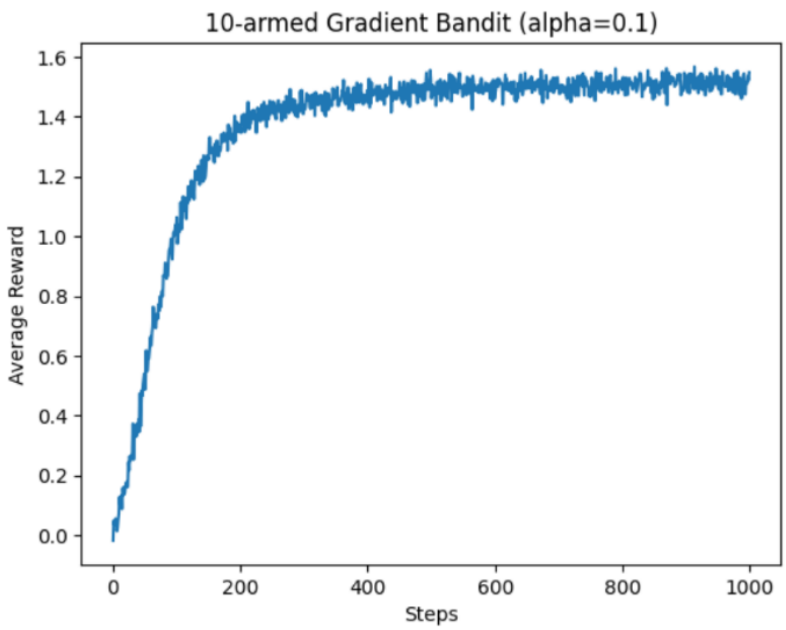
exp_preferences = np.exp(self.preferences - np.max(self.preferences))

self.probabilities = exp_preferences / np.sum(exp_preferences) return self.probabilities

```
def choose_action(self): self.softmax()
    return np.random.choice(self.k, p=self.probabilities) def update(self, action,
  reward, t):
    self.average_reward += (reward - self.average_reward) / (t + 1) reward_baseline = reward -
    self.average_reward
    for a in range(self.k): if a ==
        action:
            self.preferences[a] += self.alpha * (reward_baseline) * (1
- self.probabilities[a])
        else:
            self.preferences[a] -= self.alpha * (reward_baseline) * self.probabilities[a]
def simulate_gradient_bandit(k=10, alpha=0.1, runs=2000, steps=1000): avg_rewards = np.zeros(steps)
for run in range(runs):
    bandit = GradientBandit(k=k, alpha=alpha) q_true =
    np.random.normal(0, 1, k) rewards = np.zeros(steps)
    for t in range(steps):
        action = bandit.choose_action()
        reward = np.random.normal(q_true[action], 1) rewards[t] = reward
        bandit.update(action, reward, t) avg_rewards += rewards
avg_rewards /= runs                # Average over all runs return
avg_rewards
k = 10
```

```
alpha = 0.1

runs = 2000

steps = 1000

avg_rewards = simulate_gradient_bandit(k, alpha, runs, steps) # Plotting results

plt.plot(avg_rewards) plt.xlabel("Steps")

plt.ylabel("Average Reward")

plt.title("10-armed Gradient Bandit (alpha=0.1)") plt.show()
```

OUTPUT



Implement a Game using Dynamic Programming in Reinforcement Learning

Aim

To implement a game using Dynamic Programming in Reinforcement learning.

Algorithm

1. Define states, actions, and rewards.
2. Set initial values for all states to zero.
3. For each state, compute the value for all possible actions.
4. Update the state value with the maximum expected reward.
5. Track the maximum change (delta) to check for convergence.
6. For each state, select the action that yields the highest expected value.
7. Print the final state values and optimal policy.

Program

```
import random

states = ["hungry", "safe", "dangerous", "rested", "in_shelter"] actions = ["search_food",

"build_shelter", "rest"]

rewards = {

    ("hungry", "search_food"): 100,

    ("hungry", "build_shelter"): -50,

    ("hungry", "rest"): -100,

    ("safe", "search_food"): 10,

    ("safe", "build_shelter"): 5,

    ("safe", "rest"): 100,

    ("dangerous", "search_food"): -50,

    ("dangerous", "build_shelter"): 100,

    ("dangerous", "rest"): -100,

    ("rested", "search_food"): 100,

    ("rested", "build_shelter"): 5,

    ("rested", "rest"): -50,
```

```
        ("in_shelter", "search_food"): -50,
        ("in_shelter", "build_shelter"): 0,
        ("in_shelter", "rest"): 100,
}
#     Define the state transition function def
transition(state, action):
        if action == "search_food":
                return random.choice(["hungry", "safe", "dangerous"]) elif action ==
        "build_shelter":
                return "in_shelter" if state != "dangerous" else "dangerous" elif action == "rest":
                return "rested" if state in ["safe", "in_shelter"] else "dangerous"
        return state
def value_iteration(states, actions, rewards, discount_factor=0.9, theta=1e-4):
        # Initialize values for each state values = {state: 0 for state
        in states} while True:
                delta = 0
                for state in states: v =
                        values[state]
                       action_values = []
                        for action in actions:
                                        new_state = transition(state, action)
                                reward = rewards.get((state, action), -100)                    # Use -100 for
illegal actions
                                action_values.append(reward + discount_factor * values[new_state])
```

```
                    values[state] = max(action_values) delta = max(delta,
            abs(v - values[state]))
        if delta < theta: break
    # Derive the optimal policy based on values policy = {}
    for state in states: best_action = None
        best_value = float('-inf') for action in
        actions:
            new_state = transition(state, action) reward = rewards.get((state,
            action), -100)
            value = reward + discount_factor * values[new_state] if value > best_value:
                best_value = value best_action =
                action
        policy[state] = best_action return values,
    policy
values, policy = value_iteration(states, actions, rewards) print("State values:", values)
print("Optimal policy:", policy)
```

OUTPUT

State values: {'hungry': 999.9995617881813, 'safe': 999.9996056093631, 'dangerous': 999.9995617881813, 'rested': 999.9996056093631, 'in_shelter': 999.9996450484268}

Optimal policy: {'hungry': 'search_food', 'safe': 'rest', 'dangerous': 'build_shelter', 'rested': 'search_food', 'in_shelter': 'rest'}

Implement a game using Monte Carlo Algorithm

Aim

To implement a game using Monte Carlo algorithm.

Algorithm

1. Define states, actions, and rewards,initialize action-value estimates (Q(s, a)) and policy randomly.
2. Start from a random state and follow an epsilon-greedy policy.
3. With probability ε, choose a random action (exploration),Otherwise, choose the best-known action (exploitation).
4. Observe the reward and transition to a new state.
5. Compute the discounted return (G) for each state-action pair.
6. Update the average action-value estimate
7. Improve the policy by selecting the action with the highest estimated value,return the final action-value function.

Program

```
import random

states = ["hungry", "safe", "dangerous", "rested", "in_shelter"] actions = ["search_food",

"build_shelter", "rest"]

rewards = {

    ("hungry", "search_food"): 100,

    ("hungry", "build_shelter"): -50,

    ("hungry", "rest"): -100,

    ("safe", "search_food"): 10,

    ("safe", "build_shelter"): 5,

    ("safe", "rest"): 100,

    ("dangerous", "search_food"): -50,

    ("dangerous", "build_shelter"): 100,

    ("dangerous", "rest"): -100,
```

```
        ("rested", "search_food"): 100,
        ("rested", "build_shelter"): 5,
        ("rested", "rest"): -50,
        ("in_shelter", "search_food"): -50,
        ("in_shelter", "build_shelter"): 0,
        ("in_shelter", "rest"): 100,
}
def transition(state, action): if action ==
        "search_food":
                return random.choice(["hungry", "safe", "dangerous"]) elif action ==
        "build_shelter":
                return "in_shelter" if state != "dangerous" else "dangerous" elif action == "rest":
                return "rested" if state in ["safe", "in_shelter"] else "dangerous"
        return state
def monte_carlo_control(states, actions, rewards, num_episodes=10000, epsilon=0.1, discount_factor=0.9):
        returns = {(state, action): [] for state in states for action in actions}
        values = {(state, action): 0 for state in states for action in actions}
        policy = {state: random.choice(actions) for state in states} for episode in
        range(num_episodes):
                episode_history = []
                state = random.choice(states) while True:
                        if random.uniform(0, 1) < epsilon:
                                action = random.choice(actions)                        # Explore
```

else:

action = policy[state]

new_state = transition(state, action) reward = rewards.get((state,

action), -100)

episode_history.append((state, action, reward))

if new_state == "dangerous": # End the episode if the state becomes dangerous

break

state = new_state G = 0

visited = set()

for state, action, reward in reversed(episode_history):

G = reward + discount_factor * G if (state, action)

not in visited:

returns[(state, action)].append(G)

values[(state, action)] = sum(returns[(state, action)]) / len(returns[(state, action)])

visited.add((state, action)) for state in states:

best_action = max(actions, key=lambda a: values[(state, a)]) policy[state] = best_action

return values, policy

values_mc, policy_mc = monte_carlo_control(states, actions, rewards) print("MC - Value for each (state, action):", values_mc)

print("MC - Learned policy:", policy_mc)

Output

MC - Value for each (state, action): {('hungry', 'search_food'): 180.4606623237469, ('hungry', 'build_shelter'): 180.38617390546656,
('hungry', 'rest'): -100.0, ('safe', 'search_food'): 172.68952267786784,
('safe', 'build_shelter'): 232.2234957097028, ('safe', 'rest'): 232.35186480913535, ('dangerous', 'search_food'): 135.9442955038858,
('dangerous', 'build_shelter'): 100.0, ('dangerous', 'rest'): -100.0, ('rested', 'search_food'): 135.09279288866816, ('rested', 'build_shelter'): 134.91371552375423, ('rested', 'rest'): -50.0, ('in_shelter', 'search_food'): 95.13183068227765, ('in_shelter', 'build_shelter'): 235.57718892425146, ('in_shelter', 'rest'): 235.75923087050336}

MC - Learned policy: {'hungry': 'search_food', 'safe': 'rest', 'dangerous': 'search_food', 'rested': 'search_food', 'in_shelter': 'rest'}

Implementation of SARSA

Aim

To implement SARSA

Algorithm

1. Define the environment,and initialize Q-table (default values = 0).
2. Set hyperparameters: alpha (learning rate), gamma (discount factor), and epsilon (exploration rate).
3. Start from an initial state and choose an action using epsilon-greedy.
4. Take the action, observe the reward and next state.
5. Choose the next action using the policy.
6. Apply SARSA update,and repeat until the episode ends.
7. Run the learned policy over multiple episodes and compute the average reward.

Program

```
import numpy as np

if not hasattr(np, 'bool8'): np.bool8 = np.bool_

import gym

from collections import defaultdict class SARSAAgent:

    def __init__(self, env, alpha=0.1, gamma=0.99, epsilon=0.1): self.env = env

        self.alpha = alpha self.gamma =

        gamma self.epsilon = epsilon

        self.q_table = defaultdict(float) def

    choose_action(self, state):
```

if np.random.rand() < self.epsilon: return

self.env.action_space.sample()

else:

q_values = [self.q_table[(state, action)] for action in range(self.env.action_space.n)]

max_q = max(q_values)

return np.random.choice([action for action, q in enumerate(q_values) if q == max_q])

def learn(self, state, action, reward, next_state, next_action):

td_target = reward + self.gamma * self.q_table[(next_state, next_action)]

td_error = td_target - self.q_table[(state, action)] self.q_table[(state, action)] +=

self.alpha * td_error

def train(self, episodes): rewards = []

for episode in range(episodes): result =

self.env.reset()

state = result[0] if isinstance(result, tuple) else result action = self.choose_action(state)

total_reward = 0 done =

False while not done:

step_result = self.env.step(action) if len(step_result)

== 5:

next_state, reward, terminated, truncated, _ =
step_result

done = terminated or truncated else:

next_state, reward, done, _ = step_result

next_action = self.choose_action(next_state) self.learn(state, action, reward, next_state,

next_action) state, action = next_state, next_action

total_reward += reward

rewards.append(total_reward) if (episode

+ 1) % 100 == 0:

print(f"Episode {episode + 1}/{episodes} - Total Reward: {total_reward}")

return rewards

def evaluate(self, episodes): total_rewards = 0

for episode in range(episodes): result =

self.env.reset()

state = result[0] if isinstance(result, tuple) else result done = False

while not done:

action = self.choose_action(state) step_result =

self.env.step(action) if len(step_result) == 5:

next_state, reward, terminated, truncated, _ = step_result

done = terminated or truncated else:

next_state, reward, done, _ = step_result total_rewards += reward

state = next_state

average_reward = total_rewards / episodes print(f"Average Reward over

{episodes} episodes: {average_reward}")

```
        return average_reward
env = gym.make("Taxi-v3", render_mode="rgb_array") agent =
SARSAAgent(env)
training_rewards = agent.train(episodes=1000)
agent.evaluate(episodes=100) plt.plot(training_rewards)
plt.xlabel('Episode')
plt.ylabel('Total Reward')
plt.title('Training Rewards over Episodes (SARSA)') plt.show()
```

Output

```
Episode 100/1000 - Total Reward: -272
Episode 200/1000 - Total Reward: -122
Episode 300/1000 - Total Reward: -48
Episode 400/1000 - Total Reward: -160
Episode 500/1000 - Total Reward: -183
Episode 600/1000 - Total Reward: -112
Episode 700/1000 - Total Reward: -24
Episode 800/1000 - Total Reward: -27
Episode 900/1000 - Total Reward: 11
Episode 1000/1000 - Total Reward: -13 Average Reward over
100 episodes: -162.15
```

Training Rewards over Episodes (SARSA)
Total Reward
Episode
0
−100
−200
−300
−400
−500
−600
−700
−800
0
200
400
600
800
1000

Implementation of Q-Learning

Aim

To implement Q-learning.

Algorithm

1. Initialize Environment & Hyperparameters
2. With probability epsilon, chooses a random action (exploration).
3. Otherwise, selects the best known action from the q_table (exploitation).
4. Update Q-value formula.
5. Tracks penalties (negative rewards for illegal moves).
6. Ends episode if done=True.

Program

```
import numpy as np

if not hasattr(np, 'bool8'): np.bool8 = np.bool_

import gym import

random

from IPython.display import clear_output from time import

sleep

env = gym.make('Taxi-v3') alpha = 0.1

gamma = 0.6

epsilon = 0.1

num_episodes = 1000

q_table = np.zeros([env.observation_space.n, env.action_space.n]) for episode in

range(num_episodes):

    result = env.reset()
```

```
    state = result[0] if isinstance(result, tuple) else result                    #Handles
new API returning a tuple

    epochs, penalties, reward = 0, 0, 0 done = False

    while not done:

        if random.uniform(0, 1) < epsilon:

            action = env.action_space.sample()                # Explore action space else:

            action = np.argmax(q_table[state])                # Exploit learned values step_result

        = env.step(action)

        if len(step_result) == 5:

            next_state, reward, terminated, truncated, info = step_result done = terminated or truncated

        else:

            next_state, reward, done, info = step_result old_value =

        q_table[state, action]

        next_max = np.max(q_table[next_state])

        new_value = (1 - alpha) * old_value + alpha * (reward + gamma * next_max)

        q_table[state, action] = new_value if reward == -

        10:

            penalties += 1 state =

        next_state epochs += 1

print("Training finished.\n") total_epochs,

total_penalties = 0, 0

episodes = 100

for _ in range(episodes):
```

```
    result = env.reset()
    state = result[0] if isinstance(result, tuple) else result epochs, penalties, reward = 0, 0, 0
    done = False while not
    done:
        action = np.argmax(q_table[state]) step_result =
        env.step(action)
        if len(step_result) == 5:
            state, reward, terminated, truncated, info = step_result done = terminated or
            truncated
        else:
            state, reward, done, info = step_result if reward == -10:
            penalties += 1
        epochs += 1 total_penalties +=
    penalties total_epochs += epochs
print(f"Results after {episodes} episodes:")
print(f"Average timesteps per episode: {total_epochs / episodes}") print(f"Average penalties per episode:
{total_penalties / episodes}")
```

Output

Training finished.

Results after 100 episodes:

Average timesteps per episode: 180.94 Average penalties per

episode: 0.0

Solving a real-world problem using Q-Learning with Linear Function Approximation

Problem description: A taxi operates in a 5×5 grid world with designated pick-up and drop-off locations. The goal is to learn an optimal policy to minimize the time taken for passenger transport using Q-Learning with Linear Function Approximation, which generalizes across large state spaces without using a Q-table.

Aim

To solve a real-world problem using Q-learning with linear function approximation.

Algorithm

1. Set hyperparameters, learning rate, Discount factor, Exploration rate and decay factor.
2. Initialize weights for each action a to zero.
3. Define feature extraction function (e.g., one-hot encoding).
4. Choose action using ε-greedy policy.
5. Take action and observer next state, reward.
6. Update weight.
7. Decay exploration rate.

Program:

```
import numpy as np

if not hasattr(np, 'bool8'): np.bool8 = np.bool_

import gym

class LinearQAgent:

    def __init__(self, state_size, epsilon=1.0,    action_size,    alpha=0.01, gamma=0.9,
epsilon_decay=0.995):

        self.state_size = state_size self.action_size =

        action_size self.alpha = alpha

        self.gamma = gamma
```

self.epsilon = epsilon self.epsilon_decay =

epsilon_decay

self.weights = np.zeros((action_size, state_size)) def featurize_state(self,

state):

feature_vector = np.zeros(self.state_size) feature_vector[state] = 1

return feature_vector

def get_q_value(self, state, action): features =

self.featurize_state(state)

return np.dot(self.weights[action], features) def select_action(self,

state):

if np.random.rand() < self.epsilon:

return np.random.choice(self.action_size) else:

q_values = [self.get_q_value(state, a) for a in range(self.action_size)]

return np.argmax(q_values)

def update(self, state, action, reward, next_state, done): features = self.featurize_state(state)

q_value = self.get_q_value(state, action) if done: target =

reward

else:

next_q_values = [self.get_q_value(next_state, a) for a in range(self.action_size)]

target = reward + self.gamma * np.max(next_q_values) error = target - q_value

self.weights[action] += self.alpha * error * features if self.epsilon > 0.1:

self.epsilon *= self.epsilon_decay env =

gym.make("Taxi-v3", render_mode="ansi") state_size =

env.observation_space.n action_size = env.action_space.n

agent = LinearQAgent(state_size, action_size) episodes = 500 rewards = []

for episode in range(episodes): state =

env.reset()[0] total_reward = 0

while True:

action = agent.select_action(state) step_result =

env.step(action)

if len(step_result) == 5:

next_state, reward, terminated, truncated, _ = step_result done = terminated or truncated

else:

next_state, reward, done, _ = step_result agent.update(state, action, reward,

next_state, done) state = next_state

total_reward += reward if done:

break

rewards.append(total_reward) if (episode

+ 1) % 500 == 0:

print(f"Episode {episode + 1}, Average Reward (last 500 episodes): {np.mean(rewards[-500:])}") state =

env.reset()[0] print(env.render())

for _ in range(10):

```
action = agent.select_action(state) step_result =
env.step(action)
if len(step_result) == 5:
    next_state, reward, terminated, truncated, _ = step_result done = terminated or truncated
else:
    next_state, reward, done, _ = step_result state = next_state
print(env.render()) if done:
break
```

Output:

```
Episode 500, Average Reward (last 500 episodes): -276.646
Episode 1000, Average Reward (last 500 episodes): -235.874
Episode 1500, Average Reward (last 500 episodes): -216.636
Episode 2000, Average Reward (last 500 episodes): -194.244
Episode 2500, Average Reward (last 500 episodes): -164.772
Episode 3000, Average Reward (last 500 episodes): -139.082
Episode 3500, Average Reward (last 500 episodes): -119.112
Episode 4000, Average Reward (last 500 episodes): -105.826
Episode 4500, Average Reward (last 500 episodes): -85.752
Episode 5000, Average Reward (last 500 episodes): -71.252
+ ----------+
|R: | : :G|
| : | : : |
| : : : : |
| | : | : |
|Y| : |B: |
```

```
+ ----------------+
+ ----------------+
|R: | : :G|
| : | : : |
| : : : : |
| | : | : |
|Y| : |B: |
+ ----------------+
    (West)
+ ----------------+
|R: | : :G|
| : | : : |
| : : : : |
| | : | : |
|Y| : |B: |
+ ----------------+
    (West)
+ ----------------+
|R: | : :G|
| : | : : |
| : : : : |
| | : | : |
|Y| : |B: |
+ ----------------+
    (West)
+ ----------------+
|R: | : :G|
```

```
| : | : : |
| : : : : |
| | : | : |
|Y| : |B: |
+ ----------------+
    (West)
+ ----------------+
|R: | : :G|
| : | : : |
| : : : : |
| | : | : |
|Y| : |B: |
+ ----------------+
    (West)
+ ----------------+
|R: | : :G|
| : | : : |
| : : : : |
| | : | : |
|Y| : |B: |
+ ----------------+
    (West)
+ ----------------+
|R: | : :G|
| : | : : |
| : : : : |
| | : | : |
```

|Y| : |B: |

+ ----------+

(West)

+ ----------+

|R: | : :G|

| : | : : |

| : : : : |

| | : | : |

|Y| : |B: |

+ ----------+

(West)

+ ----------+

|R: | : :G|

| : | : : |

| : : : : |

| | : | : |

|Y| : |B: |

+ ----------+

(West)

+ ----------+

|R: | : :G|

| : | : : |

| : : : : |

| | : | : |

|Y| : |B: |

+ ----------+

(West)

Implement SARSA with Linear Function Approximation

Aim:

To implement SARSA with linear function approximation.

Algorithm:

1. Set hyperparameters,learning rate,discount factor,exploration rate.
2. Initialize weights randomly for all state-action pairs.
3. Define feature extraction function for state-action representation.
4. Choose an action using ε-greedy policy.
5. Repeat until the episode ends.
6. Update weights using gradient descent.
7. Update state and action.
8. End episode and store the total reward.

Program:

```
import numpy as np

if not hasattr(np, 'bool8'): np.bool8 = np.bool_

import gym

class LinearSARSA:

    def __init__(self, state_dim, epsilon=0.1):      action_dim,          alpha=0.01, gamma=0.99,

        self.state_dim = state_dim

        self.action_dim = action_dim self.alpha =

        alpha self.gamma = gamma self.epsilon =

        epsilon

        self.weights = np.random.rand(state_dim * action_dim) def

    featurize_state_action(self, state, action):

        features = np.zeros(self.state_dim * self.action_dim)
```

```
        features[action * self.state_dim:(action + 1) * self.state_dim] =
state

        return features

    def q_value(self, state, action):

        features = self.featurize_state_action(state, action) return np.dot(self.weights,

        features)

    def select_action(self, state):

        if np.random.rand() < self.epsilon:

            return np.random.choice(self.action_dim) else:

            q_values = [self.q_value(state, a) for a in range(self.action_dim)]

            return np.argmax(q_values)

    def update(self, state, action, reward, next_state, next_action): current_q = self.q_value(state, action)

        next_q = self.q_value(next_state, next_action) target = reward +

        self.gamma * next_q

        td_error = target - current_q

        self.weights += self.alpha * td_error * self.featurize_state_action(state, action)

env = gym.make("CartPole-v1")

state_dim = env.observation_space.shape[0] action_dim =

env.action_space.n

agent = LinearSARSA(state_dim, action_dim) num_episodes

= 50

for episode in range(num_episodes): state = env.reset()

    state = state[0] if isinstance(state, tuple) else state action = agent.select_action(state)
```

```
total_reward = 0 while
True:
    step_result = env.step(action) if
    len(step_result) == 5:
        next_state, reward, terminated, truncated, _ = step_result done = terminated or truncated
    else:
        next_state, reward, done, _ = step_result next_action =
    agent.select_action(next_state)
    agent.update(state, action, reward, next_state, next_action) state, action = next_state, next_action
    total_reward += reward if done:
        break
print(f"Episode {episode + 1}: Total Reward = {total_reward}") env.close()
```

Output:

```
Episode 1: Total Reward = 26.0
Episode 2: Total Reward = 33.0
Episode 3: Total Reward = 48.0
Episode 4: Total Reward = 42.0
Episode 5: Total Reward = 29.0
Episode 6: Total Reward = 55.0
Episode 7: Total Reward = 74.0
Episode 8: Total Reward = 55.0
Episode 9: Total Reward = 44.0
Episode 10: Total Reward = 56.0
```

Episode 11: Total Reward = 107.0

Episode 12: Total Reward = 75.0

Episode 13: Total Reward = 56.0

Episode 14: Total Reward = 84.0

Episode 15: Total Reward = 44.0

Episode 16: Total Reward = 57.0

Episode 17: Total Reward = 73.0

Episode 18: Total Reward = 86.0

Episode 19: Total Reward = 70.0

Episode 20: Total Reward = 109.0

Episode 21: Total Reward = 44.0

Episode 22: Total Reward = 70.0

Episode 23: Total Reward = 48.0

Episode 24: Total Reward = 78.0

Episode 25: Total Reward = 86.0

Episode 26: Total Reward = 54.0

Episode 27: Total Reward = 56.0

Episode 28: Total Reward = 49.0

Episode 29: Total Reward = 59.0

Episode 30: Total Reward = 105.0

Episode 31: Total Reward = 51.0

Episode 32: Total Reward = 146.0

Episode 33: Total Reward = 102.0

Episode 34: Total Reward = 75.0

Episode 35: Total Reward = 73.0

Episode 36: Total Reward = 45.0

Episode 37: Total Reward = 53.0

Episode 38: Total Reward = 69.0

Episode 39: Total Reward = 44.0

Episode 40: Total Reward = 105.0

Episode 41: Total Reward = 154.0

Episode 42: Total Reward = 97.0

Episode 43: Total Reward = 56.0

Episode 44: Total Reward = 99.0

Episode 45: Total Reward = 60.0

Episode 46: Total Reward = 79.0

Episode 47: Total Reward = 145.0

Episode 48: Total Reward = 72.0

Episode 49: Total Reward = 67.0

Episode 50: Total Reward = 45.0

Implement a game using Deep Reinforcement Learning

Description: This project implements a game where an AI agent learns to navigate an environment using Deep Reinforcement Learning (DRL). The agent is trained using the Deep Q-Network (DQN) algorithm to maximize rewards by making optimal decisions.

Aim

To implement a game using Deep reinforcement learning.

Algorithm

1. Set hyperparameters,discount factor,learning rate,batch size,exploration rate,target update frequency.
2. Initialize policy network,and target network with the same weights.
3. Create a Replay Buffer to store experiences.
4. Reset environment and get initial state s.
5. Select action and using ε-greedy policy.
6. Take action and observe next state and reward and Store experience in reply buffer.
7. Sample a batch from replay buffer if buffer size > batch size.
8. Update target network every few episodes and decay exploration rate ϵ.

Program

```
import gymnasium as gym import

numpy as np import random

import torch

import torch.nn as nn import

torch.optim as optim

from collections import deque

device = torch.device("cuda" if torch.cuda.is_available() else "cpu") class DQN(nn.Module):

    def __init__(self, input_dim, output_dim):

        super(DQN, self).__init__()

        self.fc1 = nn.Linear(input_dim, 64)
```

```
        self.fc2 = nn.Linear(64, 64) self.fc3 = nn.Linear(64,
        output_dim)

    def forward(self, x):
        x = torch.relu(self.fc1(x)) x =
        torch.relu(self.fc2(x)) return self.fc3(x)
class ReplayBuffer:
    def __init__(self, capacity): self.buffer =
        deque(maxlen=capacity)
    def push(self, state, action, reward, next_state, done): self.buffer.append((state, action, reward,
        next_state, done))
    def sample(self, batch_size):
        return random.sample(self.buffer, batch_size) def __len__(self):
        return len(self.buffer) GAMMA =
0.99
LEARNING_RATE = 1e-3 BATCH_SIZE
= 32
EPSILON_DECAY = 0.995
MIN_EPSILON = 0.01
TARGET_UPDATE_FREQUENCY = 10
env = gym.make("CartPole-v1")
n_state = env.observation_space.shape[0] n_action =
env.action_space.n
policy_net = DQN(n_state, n_action).to(device) target_net = DQN(n_state,
n_action).to(device)
```

```
target_net.load_state_dict(policy_net.state_dict()) target_net.eval()

optimizer = optim.Adam(policy_net.parameters(), lr=LEARNING_RATE) replay_buffer =

ReplayBuffer(10000)


def select_action(state, epsilon): if random.random()

        < epsilon:

                return env.action_space.sample() else:

                state = torch.tensor(state, dtype=torch.float32).unsqueeze(0).to(device)

                return torch.argmax(policy_net(state)).item() num_episodes = 20

epsilon = 1.0

for episode in range(num_episodes): state, _ =

        env.reset() total_reward = 0

        done = False while not

        done:

                action = select_action(state, epsilon)

                next_state, reward, terminated, truncated, _ = env.step(action) done = terminated or truncated

                replay_buffer.push(state, action, reward, next_state, done) state = next_state

                total_reward += reward

                if len(replay_buffer) > BATCH_SIZE:

                        batch = replay_buffer.sample(BATCH_SIZE)

                        states, actions, rewards, next_states, dones = zip(*batch)
```

```
                states = torch.tensor(states, dtype=torch.float32).to(device) actions = torch.tensor(actions,

                dtype=torch.int64).to(device)

                rewards = torch.tensor(rewards, dtype=torch.float32).to(device)

                next_states = torch.tensor(next_states, dtype=torch.float32).to(device)

                dones = torch.tensor(dones, dtype=torch.float32).to(device)


                q_values = policy_net(states).gather(1, actions.unsqueeze(1)).squeeze(1)

                next_q_values = target_net(next_states).max(1)[0]

                target_q_values = rewards + GAMMA * next_q_values * (1 -
dones)


                loss = nn.MSELoss()(q_values, target_q_values.detach()) optimizer.zero_grad()

                loss.backward()

                torch.nn.utils.clip_grad_norm_(policy_net.parameters(), max_norm=1.0)

                optimizer.step()

        if episode % TARGET_UPDATE_FREQUENCY == 0:

            target_net.load_state_dict(policy_net.state_dict()) epsilon =

        max(MIN_EPSILON, epsilon * EPSILON_DECAY)

        print(f"Episode {episode}, Total Reward: {total_reward}, Epsilon:
{epsilon:.2f}")

env.close()
```

Output

Episode 0, Total Reward: 32.0, Epsilon: 0.99

Episode 1, Total Reward: 14.0, Epsilon: 0.99

Episode 2, Total Reward: 9.0, Epsilon: 0.99

Episode 3, Total Reward: 18.0, Epsilon: 0.98

Episode 4, Total Reward: 19.0, Epsilon: 0.98

Episode 5, Total Reward: 34.0, Epsilon: 0.97

Episode 6, Total Reward: 17.0, Epsilon: 0.97

Episode 7, Total Reward: 15.0, Epsilon: 0.96

Episode 8, Total Reward: 19.0, Epsilon: 0.96

Episode 9, Total Reward: 15.0, Epsilon: 0.95

Episode 10, Total Reward: 11.0, Epsilon: 0.95

Episode 11, Total Reward: 25.0, Epsilon: 0.94

Episode 12, Total Reward: 10.0, Epsilon: 0.94

Episode 13, Total Reward: 9.0, Epsilon: 0.93

Episode 14, Total Reward: 14.0, Epsilon: 0.93

Episode 15, Total Reward: 14.0, Epsilon: 0.92

Episode 16, Total Reward: 11.0, Epsilon: 0.92

Episode 17, Total Reward: 11.0, Epsilon: 0.91

Episode 18, Total Reward: 21.0, Epsilon: 0.91

Episode 19, Total Reward: 32.0, Epsilon: 0.90

Implement Deep Q-Learning in Python using Keras & OpenAI Gym for an Agent that can play a Game

Description: This project implements a game where an AI agent learns to navigate an environment using Deep Reinforcement Learning (DRL). The agent is trained using the Deep Q-Network (DQN) algorithm implemented with Keras and OpenAI Gym to maximize rewards by making optimal decisions.

Aim

To develop an AI agent capable of learning and improving its gameplay through reinforcement learning techniques, specifically using Deep
Q-Learning with Keras and OpenAI Gym.

Algorithm

1. Set hyperparameters,learning rate,discount factor,exploration rate,decay factor and minimum.
2. Initialize Q-function parameters (weights) randomly.
3. Select action using ε-greedy policy
4. Take action and observe next state,reward and compute TD target,TD error,and update weight,state and action.
5. Decay ϵ using an exponential decay to reduce exploration.
6. After training, test the model with ϵ=0.
7. Observe average episode reward over multiple trials.

Program:

```
import numpy as np

if not hasattr(np, 'bool8'): np.bool8 = np.bool_

import gym

env = gym.make("CartPole-v1")

state_size = env.observation_space.shape[0] action_size =

env.action_space.n

weights = np.random.rand(state_size, action_size) alpha = 0.01

gamma = 0.99
```

epsilon = 1.0

epsilon_decay = 0.995

min_epsilon = 0.01

num_episodes = 20

def get_q_values(state):

return np.dot(state, weights) def

select_action(state, epsilon):

if np.random.rand() < epsilon: return

env.action_space.sample()

return np.argmax(get_q_values(state)) for episode in

range(num_episodes):

state, _ = env.reset()

action = select_action(state, epsilon) total_reward = 0

done = False while not

done:

next_state, reward, terminated, truncated, _ = env.step(action) done = terminated or truncated

next_action = select_action(next_state, epsilon) if not done else
None

q_current = get_q_values(state)[action]

q_next = get_q_values(next_state)[next_action] if not done else 0 td_target = reward + gamma *

q_next

td_error = td_target - q_current

weights[:, action] += alpha * td_error * state state, action = next_state,

next_action total_reward += reward

epsilon = max(min_epsilon, epsilon * epsilon_decay)

```
    print(f"Episode {episode + 1}, Total Reward: {total_reward}, Epsilon:
{epsilon:.2f}")

env.close()
```

Output

```
Episode 1, Total Reward: 9.0, Epsilon: 0.99
Episode 2, Total Reward: 35.0, Epsilon: 0.99
Episode 3, Total Reward: 13.0, Epsilon: 0.99
Episode 4, Total Reward: 29.0, Epsilon: 0.98
Episode 5, Total Reward: 35.0, Epsilon: 0.98
Episode 6, Total Reward: 10.0, Epsilon: 0.97
Episode 7, Total Reward: 17.0, Epsilon: 0.97
Episode 8, Total Reward: 30.0, Epsilon: 0.96
Episode 9, Total Reward: 30.0, Epsilon: 0.96
Episode 10, Total Reward: 59.0, Epsilon: 0.95
Episode 11, Total Reward: 59.0, Epsilon: 0.95
Episode 12, Total Reward: 12.0, Epsilon: 0.94
Episode 13, Total Reward: 19.0, Epsilon: 0.94
Episode 14, Total Reward: 31.0, Epsilon: 0.93
Episode 15, Total Reward: 18.0, Epsilon: 0.93
Episode 16, Total Reward: 20.0, Epsilon: 0.92
Episode 17, Total Reward: 22.0, Epsilon: 0.92
Episode 18, Total Reward: 35.0, Epsilon: 0.91
Episode 19, Total Reward: 68.0, Epsilon: 0.91
Episode 20, Total Reward: 14.0, Epsilon: 0.90
```